

SMC Confluence Master v2 — Código fuente

Script TODO-EN-UNO: estrategia de backtest + alertas en vivo · Pine Script v6 · 550 líneas

ADVERTENCIA IMPORTANTE: este PDF es para LEER y guardar como copia de seguridad. **NUNCA copies el código desde el PDF** para pegarlo en TradingView: los PDF rompen las líneas largas y añaden los números de línea, y el script no compilará. Para instalarlo usa SIEMPRE el botón "Copiar código" de la página:

claude.ai/code/artifact/4b06ea60-0bc4-4995-9c9d-e733b2b079b0

Los números de línea de la izquierda sirven para localizar cualquier error que marque el Editor Pine (los errores de TradingView siempre indican línea y columna). La configuración oficial validada ya viene incluida como valores por defecto de los inputs — ver la sección 5 de la Guía.

El código también está guardado en tu repositorio de GitHub: prop-consistency-tracker → rama `claude/tradingview-pine-script-indicator-x60veq` → `tradingview/SMC_Confluence_Master_Strategy.pine`

```

1 // @version=6
2 // =====
3 // SMC CONFLUENCE MASTER v2 - BACKTEST (Pine Script v6)
4 // VERSION DEFINITIVA TODO-EN-UNO: backtest en el Probador + alertas
5 // en vivo con alert() en las entradas (mismas operaciones exactas).
6 // Comision 2.5 USD por contrato y lado, slippage 1 tick.
7 // Sweep + MSS/CHoCH + FVG + Sesion + HTF + VWAP + Volumen + SMT + EQH/EQL
8 // Diseñado para ES / NQ en 15m y 1H (tambien funciona en 1m-5m).
9 // Archivo 100% ASCII para evitar problemas de codificacion al copiar.
10 //
11 // COMO USARLO:
12 // 1) TradingView -> Editor Pine -> pega todo -> "Anadir al grafico".
13 // 2) Usalo en 15m o 1H. El TF mayor se ajusta solo (15m->1H, 1H->4H).
14 // 3) La senal sigue el modelo completo en 3 pasos:
15 // PASO 1 Barrido de liquidez (swing/EQH/EQL con rechazo)
16 // PASO 2 MSS/CHoCH: el precio rompe la estructura menor en contra
17 // del barrido (confirma que fue sweep y no continuacion)
18 // PASO 3 Entrada al confirmar el MSS (opcional: exigir ademas
19 // el retorno al FVG, modo estricto, en el grupo 8)
20 // Todo dentro de tu ventana horaria y con score de confluencia minimo.
21 // 4) Alertas: "Crear alerta" -> condicion: este indicador ->
22 // "Cualquier funcion alert()" para recibir el mensaje con E/SL/TP.
23 //
24 // SCORE (max 7): 2 (barrido) + HTF + VWAP + Volumen + SMT + EQH/EQL.
25 // Si un filtro esta apagado no penaliza. EQH/EQL es bonus.
26 //
27 // MODO DE RIESGO (grupo 8b):
28 // - "ATR (estructura)": SL en el extremo del barrido, TP por multiples de R.
29 // - "USD fijo": perdida maxima y objetivo en dolares (usa el valor del
30 // punto del contrato y el numero de contratos). Con el filtro activado
31 // solo entra cuando el SL estructural cabe en la perdida maxima.
32 //
33 // ANTI-REPINTADO:
34 // - Senales solo al cierre de vela (barstate.isconfirmed).
35 // - Sesgo HTF con vela HTF CERRADA (idiom [1] + lookahead_on).
36 // - Pivotes confirmados con retardo fijo, nunca se redibujan.
37 // =====
38 strategy("SMC Confluence Master v2 Backtest", "SMC BT",
39 overlay=true, initial_capital=25000,
40 margin_long=0, margin_short=0,
41 commission_type=strategy.commission.cash_per_contract,
42 commission_value=2.5, slippage=1,
43 process_orders_on_close=true, fill_orders_on_standard_ohlc=true,
44 max_boxes_count=100, max_labels_count=300, max_lines_count=300)
45
46 // ===== 1) FILTRO HORARIO (America/Chicago) =====
47 grpS = "1) Filtro horario (hora CT)"
48 useSes = input.bool(true, "Senales solo en sesiones activas", group=grpS)
49 sesNY = input.session("0730-1400", "Sesion NY AM (tu ventana)", group=grpS)
50 useLDN = input.bool(false, "Incluir Londres", group=grpS)
51 sesLDN = input.session("0200-0500", "Sesion Londres", group=grpS)
52 avoidLun = input.bool(true, "Bloquear mediodia (lunch)", group=grpS)
53 sesLUN = input.session("1130-1300", "Lunch NY", group=grpS)
54 tzStr = input.string("America/Chicago", "Zona horaria", group=grpS)
55 showBg = input.bool(true, "Sombrear sesiones", group=grpS)
56
57 inNY = not na(time(timeframe.period, sesNY, tzStr))
58 inLDN = not na(time(timeframe.period, sesLDN, tzStr))
59 inLUN = not na(time(timeframe.period, sesLUN, tzStr))
60 sesionOK = (not useSes or inNY or (useLDN and inLDN)) and not (avoidLun and inLUN)
61
62 bgcolor(showBg and inNY ? color.new(color.teal, 93) : na, title="Fondo NY")
63 bgcolor(showBg and useLDN and inLDN ? color.new(color.blue, 94) : na, title="Fondo Londres")
64 bgcolor(showBg and avoidLun and inLUN ? color.new(color.gray, 92) : na, title="Fondo lunch")
65
66 // ===== 2) SESGO: TF MAYOR + VWAP =====
67 grpB = "2) Sesgo (TF mayor + VWAP)"
68 useHTF = input.bool(true, "Filtrar con sesgo del TF mayor", group=grpB)
69 autoHTF = input.bool(true, "TF mayor automatico (15m->1H, 1H->4H)", group=grpB)
70 htfTF = input.timeframe("60", "TF mayor manual", group=grpB)
71 htfLen = input.int(50, "EMA del TF mayor", minval=5, group=grpB)
72 useVW = input.bool(false, "Filtrar con VWAP de sesion", group=grpB)
73 showVW = input.bool(true, "Mostrar VWAP", group=grpB)
74
75 tfSec = timeframe.in_seconds()
76 htfAuto = tfSec <= 900 ? "60" : tfSec <= 3600 ? "240" : "D"
77 htfUse = autoHTF ? htfAuto : htfTF
78 // Vela HTF cerrada -> no repinta mientras la vela del TF mayor esta abierta
79 htfSrc = ta.ema(close, htfLen)
80 htfEma = request.security(syminfo.tickerid, htfUse, htfSrc[1], lookahead=barmerge.lookahead_on)
81 htfValido = tfSec < timeframe.in_seconds(htfUse)
82 htfBull = close > htfEma
83 htfBear = close < htfEma
84
85 vwapVal = ta.vwap(hlc3)
86 vwBull = close > vwapVal
87 vwBear = close < vwapVal
88 plot(showVW ? vwapVal : na, "VWAP sesion", color=color.new(color.orange, 0), linewidth=2)
89
90 // ===== 3) SWINGS Y BARRIDO DE LIQUIDEZ =====
91 grpP = "3) Barrido de liquidez"
92 pivLen = input.int(5, "Longitud de pivote (swings)", minval=2, group=grpP)
93 eqToLA = input.float(0.25, "Tolerancia EQH/EQL (x ATR)", step=0.05, group=grpP)
94 wickA = input.float(0.5, "Mecha minima de rechazo (x ATR)", step=0.1, group=grpP)
95 allow2 = input.bool(false, "Permitir barrido en 2 velas", group=grpP)
96 showSw = input.bool(true, "Mostrar ultimos swings", group=grpP)
97
98 atr = ta.atr(14)
99 ph = ta.pivohigh(high, pivLen, pivLen)
100 plv = ta.pivotlow(low, pivLen, pivLen)

```

```

101
102 var float lastPH = na
103 var float prevPH = na
104 var float lastPL = na
105 var float prevPL = na
106 if not na(ph)
107     prevPH := lastPH
108     lastPH := ph
109 if not na(plv)
110     prevPL := lastPL
111     lastPL := plv
112
113 eqhDiff = math.abs(lastPH - prevPH)
114 eqlDiff = math.abs(lastPL - prevPL)
115 eqhTag = not na(lastPH) and not na(prevPH) and eqhDiff <= eqToLA * atr
116 eqlTag = not na(lastPL) and not na(prevPL) and eqlDiff <= eqToLA * atr
117
118 // Cada swing es un segmento FIJO: nace en su pivote, se extiende hasta
119 // que lo barren y ahí se queda quieto (solo visual, no toca la logica)
120 colPhLn = color.new(color.red, 45)
121 colPLn = color.new(color.green, 45)
122 var line phLine = na
123 var line plLine = na
124 if showSw and not na(ph) and barstate.isconfirmed
125     phLine := line.new(bar_index - pivLen, ph, bar_index, ph, color=colPhLn)
126 if showSw and not na(plv) and barstate.isconfirmed
127     plLine := line.new(bar_index - pivLen, plv, bar_index, plv, color=colPLn)
128 if not na(phLine)
129     line.set_x2(phLine, bar_index)
130 if not na(plLine)
131     line.set_x2(plLine, bar_index)
132
133 upperWick = high - math.max(open, close)
134 lowerWick = math.min(open, close) - low
135 wickBearOK = upperWick >= wickA * atr
136 wickBullOK = lowerWick >= wickA * atr
137
138 // Sweep 1 vela: toma el nivel con mecha y CIERRA de vuelta dentro
139 sweepBear1 = not na(lastPH) and high > lastPH and close < lastPH and wickBearOK
140 sweepBull1 = not na(lastPL) and low < lastPL and close > lastPL and wickBullOK
141 // Sweep 2 velas: la anterior rompe y cierra fuera, esta cierra de vuelta dentro
142 sweepBear2a = allow2 and not na(lastPH) and high[1] > lastPH
143 sweepBear2 = sweepBear2a and close[1] >= lastPH and close < lastPH
144 sweepBull2a = allow2 and not na(lastPL) and low[1] < lastPL
145 sweepBull2 = sweepBull2a and close[1] <= lastPL and close > lastPL
146 sweepBear = sweepBear1 or sweepBear2
147 sweepBull = sweepBull1 or sweepBull2
148
149 // El nivel barrido deja de extenderse y queda fijo donde fue tomado
150 if sweepBear and barstate.isconfirmed
151     phLine := na
152 if sweepBull and barstate.isconfirmed
153     plLine := na
154
155 colSwBear = color.new(color.red, 20)
156 colSwBull = color.new(color.green, 20)
157 plotshape(sweepBear, title="Barrido de high", style=shape.triangledown,
158     location=location.abovebar, color=colSwBear, size=size.tiny)
159 plotshape(sweepBull, title="Barrido de low", style=shape.triangleup,
160     location=location.belowbar, color=colSwBull, size=size.tiny)
161
162 // ===== 4) SMT (divergencia con simbolo correlacionado) =====
163 grpM = "4) SMT (ES <-> NQ)"
164 useSMT = input.bool(true, "Usar SMT como confluencia", group=grpM)
165 corrSym = input.symbol("CME_MINI:NQ1!", "Simbolo correlacionado", group=grpM)
166
167 corrH = request.security(corrSym, timeframe.period, high, lookahead=barmerge.lookahead_off)
168 corrL = request.security(corrSym, timeframe.period, low, lookahead=barmerge.lookahead_off)
169 corrOK = not na(corrH) and not na(corrL)
170 cph = ta.pivohigh(corrH, pivLen, pivLen)
171 cplv = ta.pivotlow(corrL, pivLen, pivLen)
172 var float corrPH = na
173 var float corrPL = na
174 if not na(cph)
175     corrPH := cph
176 if not na(cplv)
177     corrPL := cplv
178
179 // SMT: tu simbolo barre su nivel pero el correlacionado NO barre el suyo
180 smtBearNow = useSMT and sweepBear and corrOK and not na(corrPH) and corrH < corrPH
181 smtBullNow = useSMT and sweepBull and corrOK and not na(corrPL) and corrL > corrPL
182
183 // ===== 5) VOLUMEN =====
184 grpV = "5) Volumen"
185 useVol = input.bool(false, "Usar volumen como confluencia", group=grpV)
186 volLen = input.int(20, "Media de volumen", group=grpV)
187 volMult = input.float(1.3, "Volumen minimo del barrido (x media)", step=0.1, group=grpV)
188 volSma = ta.sma(nz(volume), volLen)
189 volOKn = nz(volume) >= volSma * volMult
190 volOKp = nz(volume[1]) >= volSma[1] * volMult
191
192 // ===== 6) FVG (zona de entrada) =====
193 grpF = "6) FVG"
194 showFVG = input.bool(true, "Dibujar FVGs", group=grpF)
195 fvgMinA = input.float(0.15, "Tamano minimo del FVG (x ATR)", step=0.05, group=grpF)
196
197 bullFVG = low > high[2] and (low - high[2]) >= fvgMinA * atr
198 bearFVG = high < low[2] and (low[2] - high) >= fvgMinA * atr
199
200 fvgUpBor = color.new(color.green, 60)

```

```

201 fvgUpBg = color.new(color.green, 88)
202 fvgDnBor = color.new(color.red, 60)
203 fvgDnBg = color.new(color.red, 88)
204 fvgOffBor = color.new(color.gray, 75)
205 fvgOffBg = color.new(color.gray, 93)
206
207 var float bullFvgTop = na
208 var float bullFvgBot = na
209 var int bullFvgBar = na
210 var float bearFvgTop = na
211 var float bearFvgBot = na
212 var int bearFvgBar = na
213 var box bullBoxA = na
214 var box bearBoxA = na
215
216 if bullFVG
217     bullFvgTop := low
218     bullFvgBot := high[2]
219     bullFvgBar := bar_index
220     if showFVG
221         bullBoxA := box.new(bar_index - 2, low, bar_index, high[2], border_color=fvgUpBor, bgcolor=fvgUpBg)
222 if bearFVG
223     bearFvgTop := low[2]
224     bearFvgBot := high
225     bearFvgBar := bar_index
226     if showFVG
227         bearBoxA := box.new(bar_index - 2, low[2], bar_index, high, border_color=fvgDnBor, bgcolor=fvgDnBg)
228
229 // El FVG activo se extiende hasta que el precio lo atraviesa por completo;
230 // entonces queda gris (invalidado) y deja de ser zona de entrada
231 if not na(bullBoxA) and not na(bullFvgBot)
232     box.set_right(bullBoxA, bar_index)
233 if not na(bearBoxA) and not na(bearFvgTop)
234     box.set_right(bearBoxA, bar_index)
235
236 if not na(bullFvgBot) and close < bullFvgBot
237     bullFvgTop := na
238     bullFvgBot := na
239     bullFvgBar := na
240     if not na(bullBoxA)
241         box.set_border_color(bullBoxA, fvgOffBor)
242         box.set_bgcolor(bullBoxA, fvgOffBg)
243         bullBoxA := na
244 if not na(bearFvgTop) and close > bearFvgTop
245     bearFvgTop := na
246     bearFvgBot := na
247     bearFvgBar := na
248     if not na(bearBoxA)
249         box.set_border_color(bearBoxA, fvgOffBor)
250         box.set_bgcolor(bearBoxA, fvgOffBg)
251         bearBoxA := na
252
253 inBullFvg = not na(bullFvgBot) and low <= bullFvgTop and close >= bullFvgBot
254 inBearFvg = not na(bearFvgTop) and high >= bearFvgBot and close <= bearFvgTop
255
256 // ===== 7) CONFIRMACION MSS / CHOCH =====
257 grpC = "7) Confirmacion MSS (CHOCH)"
258 reqMSS = input.bool(true, "Exigir MSS tras el barrido (recomendado)", group=grpC)
259 mssl = input.int(2, "Longitud del pivote menor (estructura)", minval=2, group=grpC)
260 showMss = input.bool(true, "Marcar MSS en el grafico", group=grpC)
261
262 mph = ta.pivohigh(high, mssl, mssl)
263 mpl = ta.pivotlow(low, mssl, mssl)
264 var float minorPH = na
265 var float minorPL = na
266 if not na(mph)
267     minorPH := mph
268 if not na(mpl)
269     minorPL := mpl
270
271 // ===== 8) MAQUINA DE SETUP + SCORE =====
272 grpX = "8) Senal y riesgo"
273 winBars = input.int(4, "Ventana max. tras el barrido (velas)", minval=3, group=grpX)
274 reqFVG = input.bool(false, "Exigir retorno a FVG para entrar (estricto)", group=grpX)
275 minScore = input.int(5, "Score minimo (max 7)", minval=2, maxval=7, group=grpX)
276 slBuf = input.float(0.5, "Colchon del SL (x ATR)", step=0.1, group=grpX)
277 tp1R = input.float(1.5, "TP1 (multiplo de R)", step=0.1, group=grpX)
278 tp2R = input.float(2.5, "TP2 (multiplo de R)", step=0.1, group=grpX)
279
280 // Modo USD: SL y TP fijos en dolares segun el valor del punto del contrato.
281 // Con el filtro activado solo entra cuando el SL estructural (extremo del
282 // barrido + colchon) cabe dentro de la perdida maxima -> stop logico Y acotado.
283 grpR = "8b) Modo de riesgo"
284 modoSLTP = input.string("ATR (estructura)", "Modo de SL/TP", options=["ATR (estructura)", "USD fijo"], group=grpR)
285 qtyCt = input.int(2, "Contratos", minval=1, group=grpR)
286 riskUSD = input.float(400.0, "Perdida maxima (USD)", minval=1, group=grpR)
287 tgtUSD = input.float(1000.0, "Objetivo de ganancia (USD)", minval=1, group=grpR)
288 fitStr = input.bool(true, "Solo entrar si el SL estructural cabe en la perdida maxima", group=grpR)
289
290 var int swBearBar = na
291 var float swBearExt = na
292 var float swBearMssLvl = na
293 var bool swBearVol = false
294 var bool swBearSMT = false
295 var bool swBearEQ = false
296 var bool swBearMss = false
297 var bool bearFired = false
298
299 var int swBullBar = na
300 var float swBullExt = na

```

```

301 var float swBullMssLvl = na
302 var bool swBullVol = false
303 var bool swBullSMT = false
304 var bool swBulleQ = false
305 var bool swBullMss = false
306 var bool bullFired = false
307
308 var int lastBullSweepBar = na
309 var int lastBearSweepBar = na
310
311 // Expirar el setup si paso la ventana sin entrada
312 if not na(swBullBar) and (bar_index - swBullBar) > winBars
313     swBullBar := na
314     swBullMss := false
315 if not na(swBearBar) and (bar_index - swBearBar) > winBars
316     swBearBar := na
317     swBearMss := false
318
319 if sweepBear
320     swBearBar := bar_index
321     swBearExt := sweepBear2 ? math.max(high, high[1]) : high
322     swBearVol := volOKn or (sweepBear2 and volOKp)
323     swBearSMT := smtBearNow
324     swBearEQ := eqhTag
325     swBearMss := false
326     swBearMssLvl := na(minorPL) ? low : minorPL
327     bearFired := false
328     lastBearSweepBar := bar_index
329 if sweepBull
330     swBullBar := bar_index
331     swBullExt := sweepBull2 ? math.min(low, low[1]) : low
332     swBullVol := volOKn or (sweepBull2 and volOKp)
333     swBullSMT := smtBullNow
334     swBulleQ := eqlTag
335     swBullMss := false
336     swBullMssLvl := na(minorPH) ? high : minorPH
337     bullFired := false
338     lastBullSweepBar := bar_index
339
340 // MSS: tras barrer el low, el precio rompe al alza el ultimo swing menor
341 // (y viceversa). Es el CHOCH que separa un sweep real de una continuacion.
342 cMssUp = color.new(color.green, 55)
343 cMssDn = color.new(color.red, 55)
344 mssBullHit = not na(swBullBar) and not swBullMss and not na(swBullMssLvl) and close > swBullMssLvl
345 mssBearHit = not na(swBearBar) and not swBearMss and not na(swBearMssLvl) and close < swBearMssLvl
346 if mssBullHit
347     swBullMss := true
348     if showMss and barstate.isconfirmed
349         label.new(bar_index, high, "MSS", style=label.style_label_down, color=cMssUp, textcolor=color.white, size=size.tiny)
350 if mssBearHit
351     swBearMss := true
352     if showMss and barstate.isconfirmed
353         label.new(bar_index, low, "MSS", style=label.style_label_up, color=cMssDn, textcolor=color.white, size=size.tiny)
354
355 // Si un filtro esta apagado no penaliza el score. EQH/EQL es bonus.
356 sHtfB = not useHTF or htfBull ? 1 : 0
357 sVwB = not useVW or vwBull ? 1 : 0
358 sVolB = not useVol or swBullVol ? 1 : 0
359 sSmtB = not useSMT or not corrOK or swBullSMT ? 1 : 0
360 scoreBull = 2 + sHtfB + sVwB + sVolB + sSmtB + (swBulleQ ? 1 : 0)
361
362 sHtfS = not useHTF or htfBear ? 1 : 0
363 sVwS = not useVW or vwBear ? 1 : 0
364 sVolS = not useVol or swBearVol ? 1 : 0
365 sSmtS = not useSMT or not corrOK or swBearSMT ? 1 : 0
366 scoreBear = 2 + sHtfS + sVwS + sVolS + sSmtS + (swBearEQ ? 1 : 0)
367
368 winBull = not na(swBullBar) and (bar_index - swBullBar) <= winBars and not bullFired
369 winBear = not na(swBearBar) and (bar_index - swBearBar) <= winBars and not bearFired
370
371 fvgBullOK = inBullFvg and not na(bullFvgBar) and not na(swBullBar) and bullFvgBar >= swBullBar
372 fvgBearOK = inBearFvg and not na(bearFvgBar) and not na(swBearBar) and bearFvgBar >= swBearBar
373 entryBullTrig = not reqFVG or fvgBullOK
374 entryBearTrig = not reqFVG or fvgBearOK
375
376 mssBullOK = not reqMSS or swBullMss
377 mssBearOK = not reqMSS or swBearMss
378
379 usdMode = modoSLTP == "USD fijo"
380 pvQty = syminfo.pointvalue * qtyCt
381 riskPts = riskUSD / pvQty
382 tgtPts = tgtUSD / pvQty
383 slStructBull = swBullExt - slBuf * atr
384 slStructBear = swBearExt + slBuf * atr
385 riskOKBull = not usdMode or not fitStr or (close - slStructBull) <= riskPts
386 riskOKBear = not usdMode or not fitStr or (slStructBear - close) <= riskPts
387
388 gateBull = sesionOK and mssBullOK and entryBullTrig and riskOKBull
389 gateBear = sesionOK and mssBearOK and entryBearTrig and riskOKBear
390 buySig = winBull and gateBull and scoreBull >= minScore and barstate.isconfirmed
391 sellSig = winBear and gateBear and scoreBear >= minScore and barstate.isconfirmed
392
393 // Embudo de diagnostico: cuantos setups sobreviven a cada filtro
394 var int cntSweep = 0
395 var int cntMss = 0
396 var int cntListo = 0
397 var int cntSes = 0
398 var int cntIn = 0
399 if sweepBull or sweepBear
400     cntSweep += 1

```

```

401 if mssBullHit or mssBearHit
402     cntMss += 1
403 readyBull = winBull and mssBullOK and entryBullTrig and riskOKBull
404 readyBear = winBear and mssBearOK and entryBearTrig and riskOKBear
405 newReady = (readyBull and not readyBull[1]) or (readyBear and not readyBear[1])
406 if newReady
407     cntListo += 1
408     if sesionOK
409         cntSes += 1
410 if buySig or sellSig
411     cntIn += 1
412
413 f_fmt(x) => str.toString(x, format.mintick)
414
415 // ===== 9) EJECUCION DEL BACKTEST + ALERTAS =====
416 grpT = "9) Backtest"
417 useBE = input.bool(true, "Mover SL a breakeven a mitad de camino", group=grpT)
418 closeSes = input.bool(true, "Cerrar posiciones al terminar la sesion", group=grpT)
419
420 var float posEntry = na
421 var float posSL = na
422 var float posTP1 = na
423 var float posTP2 = na
424 var bool beDone = false
425
426 if buySig and strategy.position_size == 0
427     posEntry := close
428     posSL := usdMode and not fitStr ? close - riskPts : slStructBull
429     posTP1 := usdMode ? close + tgtPts : close + tp1R * (close - posSL)
430     posTP2 := usdMode ? close + tgtPts : close + tp2R * (close - posSL)
431     beDone := false
432     strategy.entry("Long", strategy.long, qty=qtyCt)
433     // salidas colocadas en la MISMA vela: el stop protege desde el primer tick
434     if usdMode
435         strategy.exit("XL", "Long", stop=posSL, limit=posTP1)
436     else
437         strategy.exit("XL1", "Long", qty_percent=50, stop=posSL, limit=posTP1)
438         strategy.exit("XL2", "Long", qty_percent=100, stop=posSL, limit=posTP2)
439     aTxtB = "SMC COMPRA " + syminfo.ticker + " (" + timeframe.period + "m)"
440     aTxtB := aTxtB + " | Score " + str.toString(scoreBull) + "/"
441     aTxtB := aTxtB + " | E " + f_fmt(close) + " SL " + f_fmt(posSL)
442     aTxtB := aTxtB + " TP1 " + f_fmt(posTP1) + " TP2 " + f_fmt(posTP2)
443     alert(aTxtB, alert.freq_once_per_bar_close)
444
445 if sellSig and strategy.position_size == 0 and not buySig
446     posEntry := close
447     posSL := usdMode and not fitStr ? close + riskPts : slStructBear
448     posTP1 := usdMode ? close - tgtPts : close - tp1R * (posSL - close)
449     posTP2 := usdMode ? close - tgtPts : close - tp2R * (posSL - close)
450     beDone := false
451     strategy.entry("Short", strategy.short, qty=qtyCt)
452     if usdMode
453         strategy.exit("XS", "Short", stop=posSL, limit=posTP1)
454     else
455         strategy.exit("XS1", "Short", qty_percent=50, stop=posSL, limit=posTP1)
456         strategy.exit("XS2", "Short", qty_percent=100, stop=posSL, limit=posTP2)
457     aTxtS = "SMC VENTA " + syminfo.ticker + " (" + timeframe.period + "m)"
458     aTxtS := aTxtS + " | Score " + str.toString(scoreBear) + "/"
459     aTxtS := aTxtS + " | E " + f_fmt(close) + " SL " + f_fmt(posSL)
460     aTxtS := aTxtS + " TP1 " + f_fmt(posTP1) + " TP2 " + f_fmt(posTP2)
461     alert(aTxtS, alert.freq_once_per_bar_close)
462
463 // Breakeven: en modo USD se activa a mitad de camino al TP;
464 // en modo ATR se activa cuando el precio toca TP1
465 beTrigL = usdMode ? posEntry + (posTP1 - posEntry) / 2 : posTP1
466 beTrigS = usdMode ? posEntry - (posEntry - posTP1) / 2 : posTP1
467
468 if strategy.position_size > 0
469     if useBE and not beDone and high >= beTrigL
470         beDone := true
471     stopL = beDone ? posEntry : posSL
472     if usdMode
473         strategy.exit("XL", "Long", stop=stopL, limit=posTP1)
474     else
475         strategy.exit("XL1", "Long", qty_percent=50, stop=stopL, limit=posTP1)
476         strategy.exit("XL2", "Long", qty_percent=100, stop=stopL, limit=posTP2)
477
478 if strategy.position_size < 0
479     if useBE and not beDone and low <= beTrigS
480         beDone := true
481     stopS = beDone ? posEntry : posSL
482     if usdMode
483         strategy.exit("XS", "Short", stop=stopS, limit=posTP1)
484     else
485         strategy.exit("XS1", "Short", qty_percent=50, stop=stopS, limit=posTP1)
486         strategy.exit("XS2", "Short", qty_percent=100, stop=stopS, limit=posTP2)
487
488 if closeSes and strategy.position_size != 0 and not inNY and not (useLDN and inLDN)
489     strategy.close_all(comment="Fin de sesion")
490
491 // ===== 10) PANEL DE ESTADO =====
492 grpD = "10) Panel"
493 showTbl = input.bool(true, "Mostrar panel de estado", group=grpD)
494 var table tbl = table.new(position.top_right, 2, 10,
495     bgcolor=color.new(color.black, 20), border_width=1,
496     border_color=color.new(color.gray, 60))
497 if showTbl and barstate.islast
498     cUp = color.new(color.green, 0)
499     cDn = color.new(color.red, 0)
500     cNe = color.new(color.silver, 0)

```

```

501 cwa = color.new(color.orange, 0)
502 setupTxt = "Esperando barrido"
503 setupCol = cNe
504 if winBull
505     mTx = swBullMss ? "MSS OK" : "MSS..."
506     fTx = reqFVG ? (inBullFvg ? " | FVG OK" : " | FVG...") : ""
507     setupTxt := "LONG " + str.tostring(scoreBull) + "/"7 | " + mTx + fTx
508     setupCol := cUp
509 if winBear
510     mTx = swBearMss ? "MSS OK" : "MSS..."
511     fTx = reqFVG ? (inBearFvg ? " | FVG OK" : " | FVG...") : ""
512     setupTxt := "SHORT " + str.tostring(scoreBear) + "/"7 | " + mTx + fTx
513     setupCol := cDn
514 bullNewer = na(lastBearSweepBar) or (not na(lastBullSweepBar) and lastBullSweepBar > lastBearSweepBar)
515 lastSwTxt = "-"
516 if not na(lastBullSweepBar) and bullNewer
517     lastSwTxt := "Low barrido hace " + str.tostring(bar_index - lastBullSweepBar)
518 else if not na(lastBearSweepBar)
519     lastSwTxt := "High barrido hace " + str.tostring(bar_index - lastBearSweepBar)
520 htfTxt = not htfValido ? "TF >= HTF !" : htfBull ? "Alcista" : "Bajista"
521 htfCol = not htfValido ? cwa : htfBull ? cUp : cDn
522 smtTxt = not usesMT ? "Off" : corrOK ? "Datos OK" : "SIN DATOS !"
523 smtCol = not usesMT ? cNe : corrOK ? cUp : cWa
524 modoTxt = (reqMSS ? "MSS" : "-") + " " + (reqFVG ? "FVG" : "directo")
525 modoTxt := modoTxt + " | min " + str.tostring(minScore)
526 if usdMode
527     modoTxt := modoTxt + " | USD -" + str.tostring(riskUSD, "#") + "/" + str.tostring(tgtUSD, "#")
528 table.cell(tbl, 0, 0, "SMC MASTER v2", text_color=color.white, text_size=size.small)
529 table.cell(tbl, 1, 0, syminfo.ticker + " " + timeframe.period, text_color=color.white, text_size=size.small)
530 table.cell(tbl, 0, 1, "Sesion", text_color=color.white, text_size=size.tiny)
531 table.cell(tbl, 1, 1, sesionOK ? "ACTIVA" : "FUERA", text_color=sesionOK ? cUp : cDn, text_size=size.tiny)
532 table.cell(tbl, 0, 2, "Sesgo TF mayor", text_color=color.white, text_size=size.tiny)
533 table.cell(tbl, 1, 2, htfTxt, text_color=htfCol, text_size=size.tiny)
534 table.cell(tbl, 0, 3, "VWAP", text_color=color.white, text_size=size.tiny)
535 table.cell(tbl, 1, 3, vwBull ? "Encima" : "Debajo", text_color=vwBull ? cUp : cDn, text_size=size.tiny)
536 table.cell(tbl, 0, 4, "Volumen", text_color=color.white, text_size=size.tiny)
537 table.cell(tbl, 1, 4, volOKn ? "Alto" : "Normal", text_color=volOKn ? cUp : cNe, text_size=size.tiny)
538 table.cell(tbl, 0, 5, "SMT", text_color=color.white, text_size=size.tiny)
539 table.cell(tbl, 1, 5, smtTxt, text_color=smtCol, text_size=size.tiny)
540 table.cell(tbl, 0, 6, "Ult. barrido", text_color=color.white, text_size=size.tiny)
541 table.cell(tbl, 1, 6, lastSwTxt, text_color=cNe, text_size=size.tiny)
542 table.cell(tbl, 0, 7, "Setup", text_color=color.white, text_size=size.tiny)
543 table.cell(tbl, 1, 7, setupTxt, text_color=setupCol, text_size=size.tiny)
544 table.cell(tbl, 0, 8, "Modo", text_color=color.white, text_size=size.tiny)
545 table.cell(tbl, 1, 8, modoTxt, text_color=cNe, text_size=size.tiny)
546 funTxt = str.tostring(cntSweep) + " > " + str.tostring(cntMss) + " > " + str.tostring(cntListo)
547 funTxt := funTxt + " > " + str.tostring(cntSes) + " > " + str.tostring(cntIn)
548 table.cell(tbl, 0, 9, "Embudo B>M>L>S>E", text_color=color.white, text_size=size.tiny)
549 table.cell(tbl, 1, 9, funTxt, text_color=cWa, text_size=size.tiny)
550

```